Lab experiment

5.Construct a scheduling program with C that selects the waiting process with the highest priority to execute next.

Aim:

To write and execute a C program that selects the waiting process with the highest priority to execute next using the Priority Scheduling algorithm.

Algorithm:

1.Start the program.
2.Read the number of processes.
3.Read the burst time and priority of each process.
4.Arrange the processes in ascending order of priority, where a smaller number indicates a higher priority
5.Calculate the waiting time of each process.
6.Calculate the turnaround time using `TAT = WT + BT`.
7.Calculate the average waiting time and average turnaround time.
8.Display the process details and calculated times.
9.Stop the program.

Coding:

```
#include <stdio.h>

int main()
{
    int n, i, j, temp;
    int pid[20], bt[20], priority[20];
    int wt[20], tat[20];
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        pid[i] = i + 1;
        printf("Enter burst time and priority of P%d: ", pid[i]);
        scanf("%d %d", &bt[i], &priority[i]);
    }

    /* Sort processes by priority */
    for (i = 0; i < n - 1; i++)
```

```
{
    for (j = i + 1; j < n; j++)
    {
        if (priority[i] > priority[j])
        {
            temp = priority[i];
            priority[i] = priority[j];
            priority[j] = temp;

            temp = bt[i];
            bt[i] = bt[j];
            bt[j] = temp;

            temp = pid[i];
            pid[i] = pid[j];
            pid[j] = temp;
        }
    }
}

wt[0] = 0;

for (i = 1; i < n; i++)
    wt[i] = wt[i - 1] + bt[i - 1];

for (i = 0; i < n; i++)
{
    tat[i] = wt[i] + bt[i];
    avg_wt += wt[i];
    avg_tat += tat[i];
}

printf("\nPID\tBT\tPriority\tWT\tTAT\n");

for (i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t\t%d\t%d\n",
           pid[i], bt[i], priority[i], wt[i], tat[i]);
}

printf("\nAverage Waiting Time = %.2f\n", avg_wt / n);
printf("Average Turnaround Time = %.2f\n", avg_tat / n);

return 0;
```

}

Output:

```
Output

Enter number of processes: 3
Enter burst time and priority of P1: 5  3
Enter burst time and priority of P2: 3  1
Enter burst time and priority of P3: 8  2

PID BT  Priority    WT  TAT
P2  3   1       0   3
P3  8   2       3   11
P1  5   3       11  16

Average Waiting Time = 4.67
Average Turnaround Time = 10.00
```

Result:

Thus, the C program for Priority Scheduling in an Operating System was successfully implemented and executed. The processes were scheduled according to their priorities, and the waiting time and turnaround time were calculated.